

Documentación Técnica Detallada del Servicio Geolocalización (gRPC)

Descripción General

El servicio **Geolocalización** es un microservicio desarrollado en **Node.js** que forma parte del sistema **Campus Universitario Inteligente (C5 Campus)**. Su función principal es recibir alertas de emergencia generadas dentro del campus, analizar sus coordenadas GPS y enriquecer la información antes de enviarla al servicio de prioridad mediante **gRPC**.

Este servicio permite ubicar el punto aproximado de una emergencia, determinar si pertenece a una zona registrada del campus y asociar datos útiles para el centro de monitoreo, como **cámaras cercanas, caseta responsable, dirección** y fuente de geolocalización.

Su función principal es:

- Recibir alertas de emergencia desde dispositivos IoT o tótems distribuidos en el campus.
- Procesar coordenadas geográficas de latitud y longitud.
- Determinar la zona del campus donde ocurre el incidente.
- Consultar información externa mediante OpenStreetMap cuando la ubicación no coincide con una zona registrada.
- Enviar la alerta enriquecida al microservicio Priority para obtener una clasificación de prioridad.
- Publicar el resultado procesado para que otros servicios del sistema puedan utilizarlo.

Tecnologías utilizadas

Para implementar el microservicio se utilizaron tecnologías orientadas a servicios distribuidos, comunicación entre procesos y procesamiento de información en tiempo real.

Tecnología	Función dentro del servicio
Node.js	Permite desarrollar la lógica principal del microservicio.
Express	Permite crear endpoints REST para consultar el estado del servicio, las zonas y la geolocalización.
Redis / ioredis	Se utiliza para recibir alertas, publicar resultados y mantener comunicación con otros servicios.
gRPC	Permite enviar la alerta al microservicio Priority y recibir la clasificación.
Protocol Buffers	Define el contrato de comunicación mediante el archivo priority.proto.
OpenStreetMap	Se usa como apoyo para geocodificación inversa cuando una

	coordenada no pertenece al mapa del campus.
Docker	Permite ejecutar el servicio en un contenedor aislado y portable.

Arquitectura general del servicio

El microservicio recibe una alerta generada por un dispositivo IoT, determina su ubicación dentro del campus y posteriormente se comunica con el microservicio Priority mediante gRPC. Con esta integración, la alerta no solo contiene coordenadas, sino también una zona, una dirección, cámaras cercanas, caseta responsable y una prioridad calculada.



Arquitectura general: el dispositivo IoT envía una alerta, Geolocalización identifica la zona y Priority clasifica la emergencia.

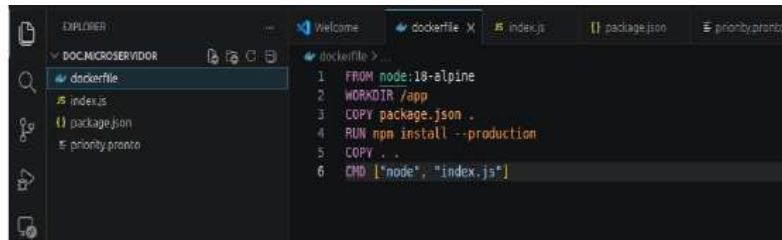
El flujo principal inicia en el dispositivo IoT y finaliza en el centro de monitoreo, donde se recibe la información enriquecida. Esto permite que el personal de seguridad tenga más contexto para atender el incidente de forma rápida.

Estructura de la carpeta de Geolocalización

El microservicio de Geolocalización se encuentra organizado en una carpeta sencilla, similar al servicio Priority. Su estructura contiene los archivos necesarios para ejecutar el servicio, instalar dependencias, definir la comunicación gRPC y procesar las alertas.

- Dockerfile
- index.js
- package.json
- priority.proto

Dockerfile



Dockerfile utilizado para construir la imagen del microservicio.

El archivo Dockerfile define el entorno donde se ejecutará el microservicio de Geolocalización. Su propósito es crear una imagen Docker que contenga Node.js, las dependencias necesarias y el código fuente de la aplicación.

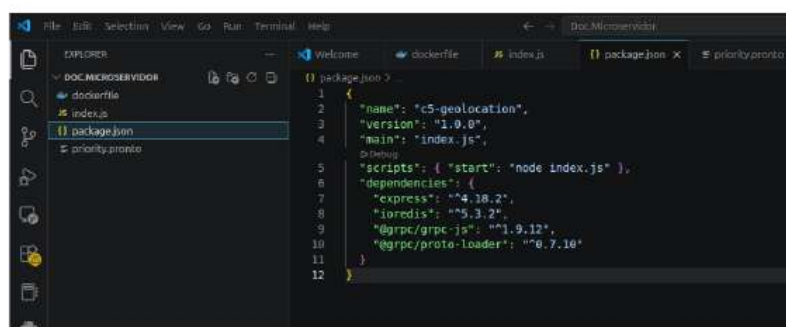
La instrucción **FROM node:18-alpine** establece la imagen base del contenedor. Se utiliza **Node.js 18** sobre Alpine Linux porque es una distribución ligera, lo que ayuda a reducir el tamaño de la imagen y permite que el servicio se inicie con menor consumo de recursos.

Posteriormente, **WORKDIR /app** establece la carpeta principal de trabajo dentro del contenedor. A partir de ese punto, las instrucciones del Dockerfile se ejecutan tomando como referencia dicha ruta.

La instrucción **COPY package.json .** copia el archivo de dependencias antes del resto del proyecto. Esto permite aprovechar la caché de Docker, ya que si no cambian las dependencias, no es necesario volver a instalarlas en cada construcción de la imagen.

Después, **RUN npm install --production** instala únicamente las dependencias necesarias para ejecutar el servicio en producción. Finalmente, **COPY . .** copia los archivos restantes y **CMD ["node", "index.js"]** inicia automáticamente la aplicación cuando el contenedor se ejecuta.

Package.json



Archivo package.json con nombre del proyecto, script de inicio y dependencias.

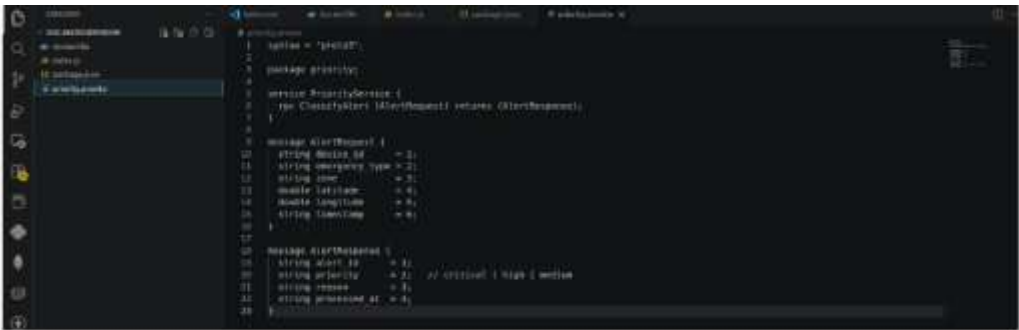
El archivo **package.json** es el archivo de configuración principal del microservicio. En él se establece la información básica del proyecto, el archivo principal que se ejecutará, los scripts disponibles y las dependencias externas necesarias para el funcionamiento del sistema.

Dentro de este archivo se define el nombre del proyecto como **c5-geolocation**, lo que permite identificarlo dentro de la arquitectura de microservicios del sistema C5 Campus. También se indica que el archivo principal de ejecución es **index.js**, por lo que Node.js inicia la aplicación desde ese punto.

La sección **scripts** define el comando **start**, el cual ejecuta **node index.js**. Esta configuración facilita iniciar el servicio tanto en un entorno local como dentro de Docker, ya que centraliza el comando de arranque de la aplicación.

En la sección **dependencies** se incluyen las bibliotecas necesarias. **Express** permite crear la API REST, **ioredis** permite la comunicación con Redis, **@grpc/grpc-js** implementa la comunicación gRPC y **@grpc/proto-loader** permite cargar el archivo **priority.proto** para interpretar los mensajes definidos con Protocol Buffers.

Priority.proto



Contrato gRPC usado para comunicarse con el microservicio Priority.

El archivo **priority.proto** es fundamental para la comunicación entre microservicios, ya que define el contrato que deben seguir tanto el cliente como el servidor **gRPC**. Gracias a este archivo, Geolocalización puede enviar datos de una alerta al servicio **Priority** y recibir una respuesta estructurada.

La instrucción **syntax = "proto3"** indica que se utiliza la versión 3 de **Protocol Buffers**. Esta tecnología permite serializar datos de manera compacta y rápida, por lo que resulta adecuada para sistemas distribuidos que requieren comunicación eficiente.

El paquete **priority** agrupa las definiciones relacionadas con el servicio de prioridad. Dentro del archivo se declara el servicio **PriorityService**, el cual expone el método **ClassifyAlert**. Este método recibe un mensaje **AlertRequest** y devuelve un mensaje **AlertResponse**.

Campo de AlertRequest	Descripción
device_id	Identificador del dispositivo o tótem que generó la alerta.

emergency_type	Tipo de emergencia reportada.
zone	Zona detectada por el microservicio de Geolocalización.
latitude	Latitud de la ubicación recibida.
longitude	Longitud de la ubicación recibida.
timestamp	Fecha y hora en que se generó la alerta.

Campo de AlertResponse	Descripción
alert_id	Identificador único de la alerta procesada.
priority	Nivel de prioridad asignado por el servicio Priority.
reason	Motivo o explicación de la clasificación.
processed_at	Fecha y hora en que la alerta fue procesada.

Index.js

```
1 // Configuración general del microservicio de Geolocalización
2 // Este archivo contiene la lógica necesaria para recibir alertas, interpretar coordenadas, determinar zonas del campus, consultar geocodificación inversa, comunicarse con Priority y publicar la alerta enriquecida.
3
4 const mongoose = require('mongoose');
5 const axios = require('axios');
6 const { Priority } = require('priority');
7 const { Geolocalizacion } = require('geolocalizacion');
8 const { Geocodificacion } = require('geocodificacion');
9 const { GeolocalizacionInversa } = require('geolocalizacion-inversa');
10 const { GeolocalizacionZonas } = require('geolocalizacion-zonas');
11 const { GeolocalizacionAlertas } = require('geolocalizacion-alertas');
12 const { GeolocalizacionPublicar } = require('geolocalizacion-publicar');
13
14 const app = express();
15 app.use(bodyParser.json());
16
17 // Configuración de mongoose
18 mongoose.connect('mongodb://localhost:27020/geolocalizacion', {
19   useNewUrlParser: true,
20   useUnifiedTopology: true,
21 });
22
23 // Definición de rutas
24 app.get('/', (req, res) => {
25   res.send('Bienvenido al microservicio de Geolocalización');
26 });
27
28 // Definición de rutas para el cliente gRPC
29 app.post('/api/geolocalizacion', (req, res) => {
30   // Lógica para recibir alertas y determinar zonas del campus
31   // ...
32 });
33
34 // Definición de rutas para la geocodificación inversa
35 app.post('/api/geocodificacion', (req, res) => {
36   // Lógica para consultar geocodificación inversa
37   // ...
38 });
39
40 // Definición de rutas para la geolocalización de zonas
41 app.post('/api/geolocalizacion-zonas', (req, res) => {
42   // Lógica para determinar zonas del campus
43   // ...
44 });
45
46 // Definición de rutas para la geolocalización de alertas
47 app.post('/api/geolocalizacion-alertas', (req, res) => {
48   // Lógica para recibir alertas y determinar zonas del campus
49   // ...
50 });
51
52 // Definición de rutas para la geolocalización de publicar
53 app.post('/api/geolocalizacion-publicar', (req, res) => {
54   // Lógica para publicar la alerta enriquecida
55   // ...
56 });
57
58 // Ejecución del servidor
59 app.listen(3000, () => {
60   console.log('Servidor ejecutando en el puerto 3000');
61 });
```

```
1 // Definición de zonas del campus
2 // Este archivo contiene la lógica necesaria para determinar las zonas del campus basadas en las coordenadas de los edificios.
3
4 const mongoose = require('mongoose');
5 const { GeolocalizacionZonas } = require('geolocalizacion-zonas');
6
7 const app = express();
8 app.use(bodyParser.json());
9
10 // Configuración de mongoose
11 mongoose.connect('mongodb://localhost:27020/geolocalizacion', {
12   useNewUrlParser: true,
13   useUnifiedTopology: true,
14 });
15
16 // Definición de rutas
17 app.get('/', (req, res) => {
18   res.send('Bienvenido al microservicio de Geolocalización');
19 });
20
21 // Definición de rutas para la geolocalización de zonas
22 app.post('/api/geolocalizacion-zonas', (req, res) => {
23   // Lógica para determinar zonas del campus
24   // ...
25 });
26
27 // Ejecución del servidor
28 app.listen(3000, () => {
29   console.log('Servidor ejecutando en el puerto 3000');
30 });
```

Primera parte de index.js: configuración general, cliente gRPC y mapa de zonas del campus.

El archivo **index.js** es el componente principal del microservicio de **Geolocalización**, ya que contiene la lógica necesaria para recibir alertas, interpretar coordenadas, determinar zonas del campus, consultar geocodificación inversa, comunicarse con **Priority** y publicar la alerta enriquecida.

Al iniciar la aplicación se importan las dependencias principales: **Express** para la API REST, **ioredis** para Redis, módulos **HTTP/HTTPS** para solicitudes externas, **gRPC** para la comunicación entre microservicios y **Proto Loader** para cargar el archivo **priority.proto**.

También se configuran variables importantes como **PORT**, **REDIS_URL** y **PRIORITY_GRPC**. Estas variables permiten definir el puerto de ejecución del servicio, la conexión con Redis y la dirección del microservicio Priority. Si no se proporcionan valores desde el entorno, el sistema utiliza valores predeterminados.

Una parte importante del archivo consiste en crear el cliente gRPC. Para ello se carga el archivo **priority.proto** con **protoLoader.loadSync** y posteriormente se crea una instancia de **PriorityService**. Este cliente permite que Geolocalización llame al método **ClassifyAlert** y reciba la prioridad asignada.

Mapa de zonas del campus

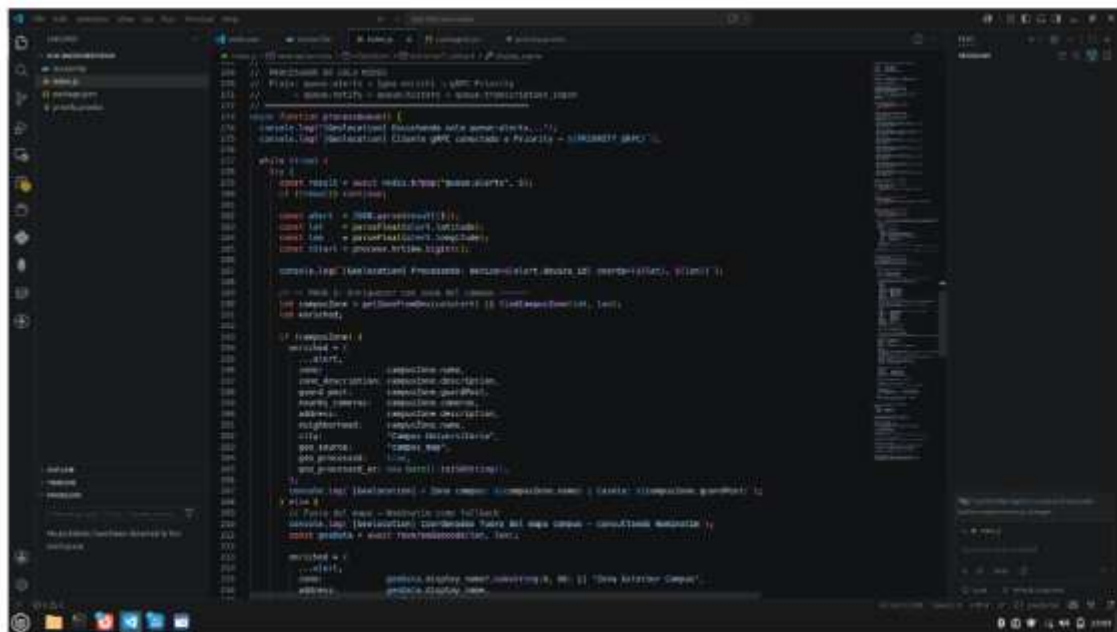
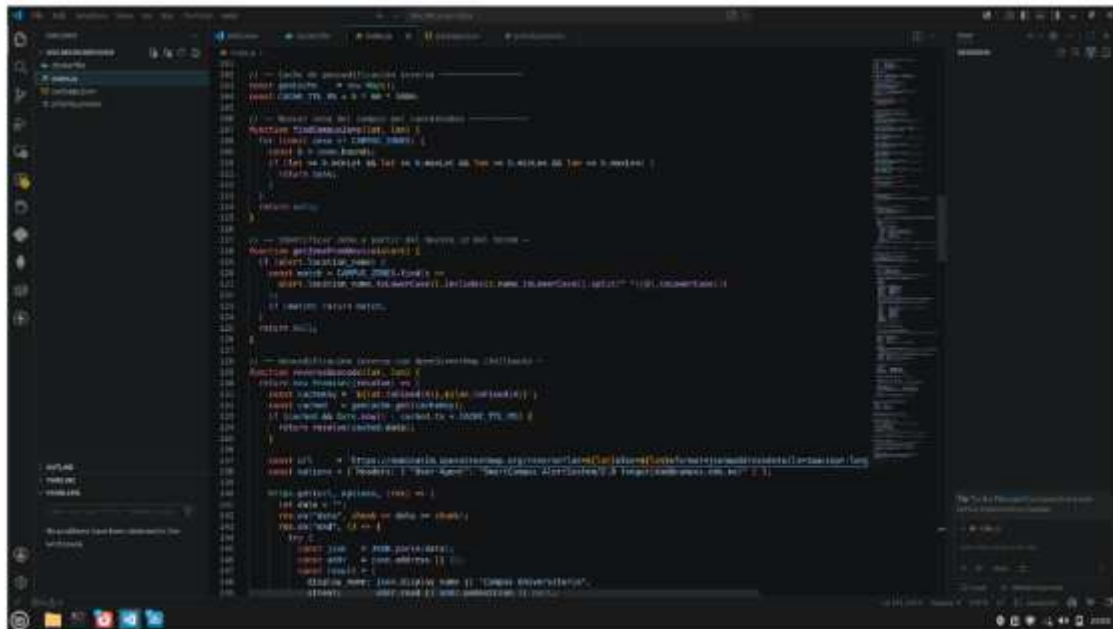
El sistema contiene un arreglo de zonas predefinidas llamado **CAMPUS_ZONES**. Cada zona representa un área del campus y contiene información útil para el centro de monitoreo, como nombre, descripción, límites geográficos, cámaras asociadas y caseta responsable.

Entre las zonas configuradas se encuentran edificios, estacionamientos, biblioteca, patio central, canchas deportivas, acceso principal y áreas administrativas. Cada una se define mediante límites de latitud y longitud, lo que permite verificar si una coordenada pertenece a dicha zona.

La función **findCampusZone(lat, lon)** compara la latitud y longitud recibidas contra los límites de cada zona. Si la coordenada se encuentra dentro de un rango válido, devuelve la zona correspondiente; si no hay coincidencia, devuelve **null** para que el sistema utilice geocodificación inversa como alternativa.

1. Recibe latitud y longitud.
2. Recorre las zonas registradas en **CAMPUS_ZONES**.
3. Compara las coordenadas contra los límites definidos.
4. Devuelve la zona encontrada o null si no pertenece al mapa interno.

Geocodificación inversa



Segunda parte de index.js: caché de geocodificación, búsqueda de zona y procesamiento de la cola de alertas.

Cuando una alerta contiene coordenadas que no coinciden con las zonas registradas del campus, el sistema utiliza la función **reverseGeocode()**. Esta función consulta **OpenStreetMap** mediante **Nominatim** para obtener información aproximada de la ubicación a partir de la latitud y longitud.

El resultado de la geocodificación inversa puede incluir nombre del lugar, dirección, calle, municipio y estado. Esta información se agrega a la alerta enriquecida para que el centro de monitoreo tenga una referencia más clara del punto donde ocurrió el incidente.

Para mejorar el rendimiento, el servicio implementa una **caché temporal** mediante un **Map**. La clave se genera a partir de las coordenadas redondeadas, y si una consulta ya fue realizada recientemente, el sistema reutiliza el resultado sin volver a solicitar datos a OpenStreetMap.

Procesamiento de alertas

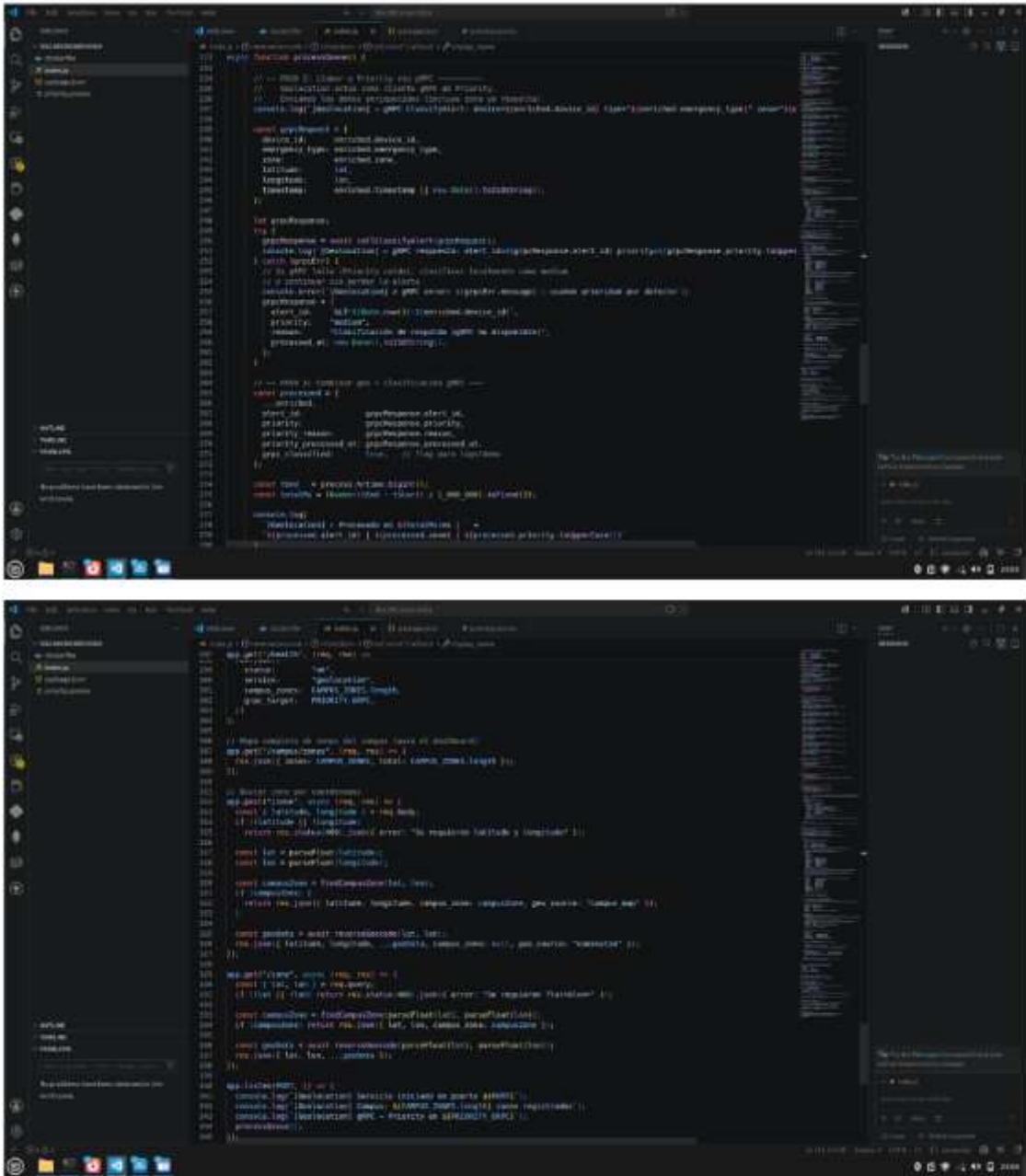
El procesamiento principal se realiza mediante una cola en **Redis**. El servicio espera alertas en la cola **queue:alerts**, las extrae, interpreta su contenido y genera una versión enriquecida con información geográfica y clasificación de prioridad.

Cuando llega una alerta, el servicio obtiene el tipo de emergencia, el identificador del dispositivo, la latitud, la longitud y la fecha. Después intenta ubicar la alerta dentro del mapa de zonas del campus. Si la zona se encuentra, se agregan datos como cámaras, caseta, descripción y fuente de geolocalización **campus map**.

Si la alerta queda fuera del mapa interno, el sistema consulta OpenStreetMap y marca la fuente como **nominatim**. De esta manera, incluso cuando el punto no está registrado previamente, la alerta conserva información útil de ubicación.

Posteriormente, el microservicio llama a Priority mediante gRPC. La función **callClassifyAlert()** envía los datos requeridos por **AlertRequest** y espera un **AlertResponse**. Si la comunicación falla, el sistema aplica una clasificación de respaldo para que la alerta no se pierda y pueda continuar el flujo de atención.

API REST y endpoints de consulta



FO-ACA-11

Versión 1

Fecha: 25/10/2018

Tercera parte de index.js: llamada gRPC, combinación de datos y endpoints REST.

Además del procesamiento de alertas, el archivo index.js expone endpoints REST que facilitan la supervisión y prueba del servicio. Estos endpoints permiten comprobar el estado del microservicio, consultar las zonas disponibles y buscar información de ubicación a partir de coordenadas.

Endpoint	Método	Función
----------	--------	---------

/health	GET	Devuelve el estado del servicio, cantidad de zonas registradas y dirección del servicio Priority.
/campus/zones	GET	Devuelve el mapa completo de zonas del campus.
/lookup	POST	Recibe latitud y longitud en el cuerpo de la petición y devuelve la zona o geocodificación.
/zone	GET	Permite buscar zona usando parámetros lat y lon en la URL.

Estos endpoints son útiles durante el desarrollo porque permiten validar que el microservicio esté activo, verificar si las coordenadas pertenecen al campus y confirmar que la integración con Priority se encuentra configurada correctamente.

Flujo de operación

El flujo de operación resume cómo se comporta el microservicio desde que se genera una alerta hasta que el centro de monitoreo recibe la información procesada.

5. Un dispositivo IoT o tótem genera una alerta de emergencia.
6. La alerta se coloca en Redis dentro de la cola correspondiente.
7. El microservicio de Geolocalización recibe la alerta y lee sus coordenadas.
8. El sistema busca la zona del campus usando findCampusZone().
9. Si no encuentra una zona, consulta OpenStreetMap mediante reverseGeocode().
10. La alerta se enriquece con zona, dirección, cámaras y caseta responsable.
11. Geolocalización envía los datos al microservicio Priority mediante gRPC.
12. Priority devuelve el nivel de prioridad y el motivo de clasificación.
13. El servicio combina la información geográfica con la prioridad recibida.
14. La alerta enriquecida se publica para que el centro de monitoreo la atienda.

Resultados obtenidos

Durante las pruebas, el microservicio permitió procesar alertas con coordenadas GPS y transformar esos datos en información más completa para el sistema C5 Campus.

- Se logró identificar correctamente zonas registradas del campus.
- Se asociaron cámaras de vigilancia cercanas a cada zona.
- Se asignó una caseta responsable para apoyar la atención del incidente.
- Se integró la comunicación con el microservicio Priority mediante gRPC.
- Se utilizó Redis para recibir y publicar información entre servicios.
- Se implementó geocodificación inversa como respaldo para ubicaciones fuera del mapa interno.
- Se mantuvo una arquitectura distribuida y modular.

Conclusión

En esta práctica se desarrolló un microservicio de **Geolocalización** capaz de recibir alertas de emergencia, analizar coordenadas GPS y determinar la zona aproximada donde ocurre un incidente dentro del campus universitario. La integración con **Redis** permitió manejar el flujo de mensajes, mientras que **gRPC** facilitó la comunicación directa con el microservicio **Priority** para obtener la clasificación de prioridad de cada alerta.

El uso de zonas predefinidas permitió enriquecer la información con cámaras, casetas y descripciones del lugar. Además, la consulta a OpenStreetMap funcionó como mecanismo de respaldo cuando las coordenadas no pertenecían a una zona registrada, evitando que la alerta quedara sin ubicación útil.

En general, el servicio demuestra cómo una arquitectura de microservicios puede mejorar la atención de emergencias dentro de un campus universitario, ya que separa responsabilidades, facilita la integración entre componentes y permite que el centro de monitoreo reciba información más precisa para tomar decisiones rápidas.